

Warum die Whisper-Transkription beim watch-Skill mit HTTP 429 fehlschlägt

Aufgetreten bei mehreren der 14 Videos in dieser Batch (u.a. [video-summary-meZirzbqXM.md](#), [video-summary-889tcWGenP0.md](#), [video-summary-AL391nkWGlc.md](#)).

Kurzantwort

Das Rate-Limit liegt an **Replicate** (dem Hosting-Layer), nicht an Whisper selbst. Ein Sub-Agent bekam die konkrete Fehlermeldung:

```
"rate limit... while you have less than $5.0 in credit"
```

Das ist eine Replicate-spezifische Kontostand-Sperre — Replicate throttelt/blockiert Accounts, deren Prepaid-Guthaben unter einer Schwelle liegt, unabhängig davon, wie oft man tatsächlich anfragt.

Root-Cause-Analyse (Code: [claude-skills/watch/scripts/whisper.py](#))

Der `watch`-Skill unterstützt drei Backends für die Transkription, in Prioritätsreihenfolge:

1. `GROQ_API_KEY` → Groq Whisper-API (`api.groq.com`)
2. `REPLICATE_API_TOKEN` → Replicate-gehostetes `openai/whisper` (`api.replicate.com`)
3. `OPENAI_API_KEY` → OpenAI Whisper-API (`api.openai.com`)

Auf diesem Rechner ist der **Replicate-Fallback-Patch** des watch-Skills aktiv (siehe Commit "Add watch skill (with Replicate whisper fallback) for sandbox sync") — vermutlich weil kein Groq/OpenAI-Key hinterlegt ist und stattdessen ein Replicate-Token genutzt wird.

Replicate ist dabei kein Whisper-Anbieter im eigentlichen Sinn, sondern eine allgemeine Modell-Hosting-Plattform, die Whisper als eines von vielen Modellen bereitstellt (`_replicate_whisper()` in `whisper.py:314`). Replicate erzwingt **kontostandbasierte Rate-Limits**: Accounts mit wenig/keinem Guthaben werden auf einer sehr niedrigen Anfragerate gedeckelt, unabhängig vom Whisper-Modell selbst — das ist Replicates eigene Plattform-Policy, keine Whisper-/OpenAI-Eigenschaft.

Zusätzlich beobachtet: Bei einigen Videos scheiterte parallel auch der **native YouTube-Untertitel-Abruf** mit HTTP 429 (z. B. bei 889tcWGenP0, AL391nkWGlc) — das ist ein komplett separates Rate-Limit von yt-dlp/YouTube selbst, nicht von Replicate, und wurde in der Analyse nicht mit dem Whisper-429 verwechselt.

Praktische Konsequenz

- Um das zu beheben: Guthaben auf dem Replicate-Account auf mindestens 5 \$ aufladen (behebt die Sperre direkt), oder
- einen `GROQ_API_KEY` hinterlegen (Groq hat laut `SKILL.md` ein großzügigeres kostenloses Kontingent und wird vom Skill ohnehin bevorzugt, wenn beide Keys vorhanden sind) — dann greift der Replicate-Fallback gar nicht erst.

- Der Skill selbst degradiert bereits automatisch auf frames-only, wenn Whisper nach 2 Rate-Limit-Versuchen weiter fehlschlägt (`MAX_429_RETRIES = 2` in `whisper.py:152`) — kein Bug, sondern beabsichtigtes Verhalten.

Zweite, unabhängige Fehlerursache: festes 6-Minuten-Verarbeitungs-Timeout (2026-08-01)

Nach Aufladen des Replicate-Guthabens auf über 5 \$ trat bei [video-summary-889tcWGenP0.md](#) (34 Min. Länge) ein **anderer** Fehler auf derselben Stelle auf: `"Replicate prediction timed out after 6 minutes"`. Das ist kein Kontostand-Problem mehr, sondern ein hartes, im Code fixtes Timeout — `_poll_replicate()` in `whisper.py:291` bricht nach 72 Poll-Versuchen à 5 s (= 6 Minuten) ab, unabhängig davon, ob Replicate die Transkription noch fertigstellen würde. Bei langen Videos (~30+ Min., ~16 MB Audio) kann die tatsächliche Verarbeitungszeit auf Replicate diese 6 Minuten überschreiten. Es gibt keinen CLI-Flag im Skill, um dieses Timeout zu verlängern.

Funktionierender Workaround — Video in Häppchen zerlegen statt am Stück transkribieren:

1. Video einmal komplett per `yt-dlp` herunterladen (gleicher Befehl wie in `download.py`)
2. Mit `ffmpeg -ss <start> -t <dauer> -c copy` in ca. 6-Minuten-Segmente schneiden (verlustfreier Stream-Copy, kein Neu-Encoding)
3. Für jedes Segment `whisper.transcribe_video()` einzeln aufrufen (Backend `replicate` erzwingen) — jedes Segment bleibt so deutlich unter dem 6-Minuten-Cap
4. Die zurückgegebenen Segment-Timestamps um den jeweiligen Chunk-Start versetzen und alle Chunks zu einem durchgehenden Transkript zusammenführen

Bei [889tcWGenP0.md](#) funktionierte das zuverlässig: 6 Chunks à ca. 5:43 Min., alle erfolgreich transkribiert, 469 Segmente insgesamt. Faustregel für zukünftige Videos: alles über ca. 15-20 Minuten Länge vorsorglich in ~6-Minuten-Häppchen zerlegen, statt erst den Direktversuch abzuwarten.

Der manuelle Chunking-Workaround skaliert nachweislich auch auf deutlich längere Videos: Bei [Lu95f1ZBlos.md](#) (134 Min.) und [WHUBWM0JoDM.md](#) (139 Min.) lief er jeweils mit 27-28 Chunks à 5 Min. fehlerfrei durch (2084 bzw. 2159 Segmente, 0 Ausfälle).

Dritte, unabhängige Fehlerursache: HTTP 413 „Payload Too Large“ (2026-09-07)

Bei beiden oben genannten Videos schlug der Direktversuch nicht mit dem 6-Minuten-Timeout fehl, sondern sofort mit `HTTP 413 (Payload Too Large)` beim Hochladen der vollen Audiodatei an Replicate — ein dritter, unabhängiger Fehlermodus zusätzlich zum kontostandbasierten 429 und zum 6-Minuten-Poll-Timeout. Betroffen war offenbar das rohe Upload-Volumen selbst (bei 134-155 Min. Videolänge entsprechend große Audiodateien), nicht die Verarbeitungsdauer. Der gleiche Workaround (Abschnitt oben: Audio vorab in ~5-Minuten-Segmente zerlegen, einzeln transkribieren, Zeitstempel versetzt zusammenführen) behebt auch diesen Fehlermodus zuverlässig — es lohnt sich also nicht, bei langen Videos zwischen 429/6-Minuten-Timeout/413 zu unterscheiden: ab ca. 15-20 Minuten Länge direkt chunken.